

Contenido: Unidad 2

Semana 5	2
2.1 Arquitectura del procesador IA-32	2
2.1.1 Modos de Operación	2
2.1.2 Espacio de direcciones	4
2.2 Ciclo de ejecución de las instrucciones	4
Semana 6	6
2.3 Registros – El modelo de Programación	6
2.4 Administración de memoria del procesador IA-32 Modo de direccionamiento Real y Protegido	10
Semana 7	12
2.5 Unidad Aritmética y Lógica – ALU	12
2.6 Unidad de Control	13
Semana 8	14
2.7 Arquitectura y representación del Juego de Instrucciones	14
2.7.2 Representación del juego de instrucciones	16
2.8 Formato de Instrucciones	18
2.8.1 Elementos que componen una instrucción	18
2.8.2 Tamaño de las instrucciones	18
2.8.3 Tipos de instrucciones	19
Semana 9	20
2.9 Fundamentos del Lenguaje Ensamblador en Sistemas GNU/Linux	20
2.9.1 Elementos básicos del lenguaje ensamblador	20
Semana 10	23
2.9 Fundamentos del Lenguaje Ensamblador en Sistemas GNU/Linux	23

2.9.1 Estructura de un programa en ensamblador.....	23
2.9.2 Directivas.....	25
2.9.3 Tipos de Instrucciones.....	27
Semana 11.....	29
2.10 Uso del Lenguaje Ensamblador con C en Sistemas GNU/Linux.....	29
2.10.1 Conceptos de programación en ensamblador y C	29
Semana 12	37
2.10 Uso del Lenguaje Ensamblador con C en Sistemas GNU/Linux.....	37
2.10.2 Programación mixta C-asm Linux.....	37
Bibliografía.....	44

Semana 5

2.1 Arquitectura del procesador IA-32.

Como se mencionó en la unidad anterior, la arquitectura IA 32 hace referencia a la familia de procesadores que iniciaron con el procesador Intel8086, el cual fue lanzado en el año 1978 como una arquitectura de 16 bits. Pasaron varios años y en 1985 se lanza el Procesador Intel386 que fue el primer procesador con una arquitectura de 32bits. Desde 1999 hasta el 2003, AMD amplio esta arquitectura de 32 bits de Intel a una de 64 bits.

2.1.1 Modos de Operación

Los procesadores IA-32 tienen cuatro modos de operación:

- Modo protegido.
- Modo de direccionamiento real.

- Modo virtual 8086.
- Modo de administración del sistema.

Irvine (2008) describe cada uno de los modos de la siguiente manera:

- ***Modo protegido:*** El modo protegido es el estado nativo del procesador, en el que están disponibles todas las instrucciones y características. Los programas reciben áreas separadas de memoria llamadas *segmentos*, y el procesador evita que los programas hagan referencia a la memoria que se encuentra fuera de sus segmentos asignados.
- ***Modo 8086 virtual:*** Mientras se encuentra en modo protegido, el procesador puede ejecutar en forma directa el software para modo de direccionamiento real, como los programas de MS-DOS, en un entorno multitarea seguro. En otras palabras, si un programa de MS-DOS falla o trata de escribir datos en el área de memoria del sistema, no afectará a los otros programas que se ejecuten al mismo tiempo. Windows XP puede ejecutar varias sesiones separadas en modo 8086 a la vez.
- ***Modo de direccionamiento real:*** Este modo implementa el entorno de programación del procesador 8086 de Intel, con unas cuantas características adicionales, como la habilidad de cambiar a otros modos. Este modo está disponible en Windows 98 y puede usarse para ejecutar un programa de MS-DOS que requiera el acceso directo a la memoria del sistema y a los dispositivos de hardware. Los programas que se ejecutan en modo de direccionamiento real pueden hacer que el sistema operativo falle (que deje de responder a los comandos).
- ***Modo de administración del sistema:*** El modo de Administración del sistema (SMM) proporciona al sistema operativo un mecanismo para implementar funciones, como la administración de energía y la seguridad del sistema. Por lo general, estas funciones las implementan los fabricantes de computadoras, quienes personalizan el procesador para una configuración específica del sistema.

2.1.2 Espacio de direcciones

Los procesadores IA-32 pueden acceder a 4 GB de memoria en modo protegido; este límite se basa en el tamaño de una dirección representada por un número entero binario sin signo, de 32 bits. Los programas en modo de direccionamiento real tienen un rango de memoria de 1 MB. Si el procesador se encuentra en modo protegido y ejecuta varios programas en modo 8086 virtual, cada programa tiene su propia área de memoria de 1 MB. (Irvine, 2008, p. 34)

2.2 Ciclo de ejecución de las instrucciones.

Cuando se habla de la ejecución de instrucciones, se deben de seguir ciertos pasos, para ello Orenga & Manonellas (2011) describen dichos pasos de la siguiente manera:

El **ciclo de ejecución** es la secuencia de operaciones que se hacen para ejecutar cada una de las instrucciones y lo dividiremos en cuatro fases principales:

- 1) Lectura de la instrucción.
- 2) Lectura de los operandos fuente.
- 3) Ejecución de la instrucción y almacenamiento del operando destino.
- 4) Comprobación de interrupciones.

Irvine (2008) contempla otra manera de describir los pasos que se llevan a cabo en el ciclo de ejecución de las instrucciones y se presentan de la siguiente manera:

- **Búsqueda:** la unidad de control busca la instrucción en la cola de instrucciones e incrementa el apuntador de instrucciones (IP). A este apuntador también se le conoce como el *contador del programa*.
- **Decodificación:** la unidad de control decodifica la función de la instrucción para determinar lo que ésta debe hacer. Los operandos de entrada de la instrucción se pasan a la unidad aritmética-lógica (ALU),

y se envían señales a la ALU para indicar la operación que se va a realizar.

- ***Búsqueda de operandos:*** si la instrucción utiliza un operando de entrada ubicado en memoria, la unidad de control utiliza una operación de *lectura* para obtener el operando y copiarlo en los registros internos. Estos registros no son visibles para los programas de los usuarios.
- ***Ejecución:*** la ALU ejecuta la instrucción, utilizando los registros con nombre y los registros internos como operandos, y envía el resultado a los registros con nombre y a la memoria. La ALU actualiza las banderas de estado que proporcionan información acerca del estado del procesador.
- ***Almacenamiento del operando del resultado:*** si el operando de resultante está en memoria, la unidad de control utiliza una operación de escritura para almacenar el dato.

En la tabla 1 se observa de manera esquemática las operaciones que se realizan en cada fase.

Tabla 1

Fases del ciclo de ejecución.

Ingeniería en Desarrollo
de Software

Inicio del ciclo de ejecución	
Fase 1 Lectura de la instrucción	Leer la instrucciones.
	Descodificar la instrucción.
	Actualizar el PC.
Fase 2 Lectura de los operandos fuentes	Calcular la direccion y leer el primer operando fuente
	Calcular la direccion y leer el segundo operando fuente
Fase 3 Ejecucion de la instrucción y almacenamiento del operando destino	Ejecutar la instrucción.
Fase 4 Comprobacion de interrupciones	Comprobar si algun dispositivo ha solicitado una interrupcion.

Nota: La tabla describe cada una de las fases del ciclo de ejecución de una instrucción. Fuente: Orenge & Manonellas (2011)

Semana 6

2.3 Registros – El modelo de Programación.

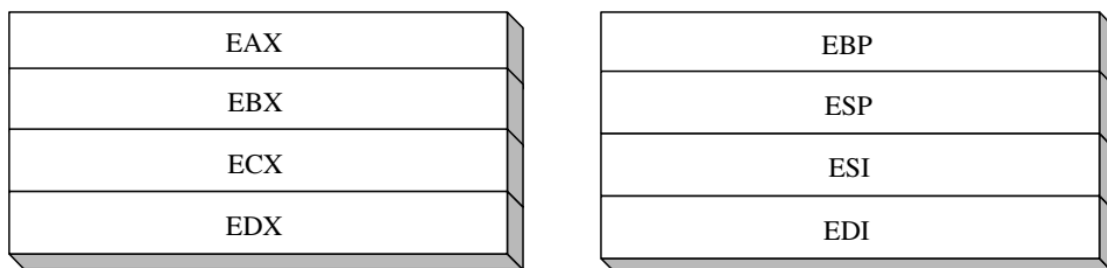
Los registros, son el mecanismo de almacenamiento del cual dispone el procesador para un acceso más rápido a los datos que se estén manipulando. Se clasifican de la siguiente manera:

- Registros de Propósito general.
- Registros de segmento
- Registro de banderas (EFLAGS) de estado y de control.
- Registro de Instrucciones.

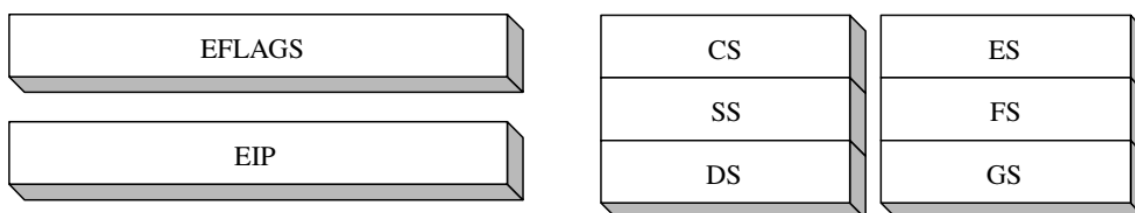
La Figura 1 muestra los registros básicos de cada tipo en arquitectura de 32 bits, la figura 2 muestra los nuevos registros agregados en una arquitectura de 64 bits:

Figura 1

Registros de propósito general de 32 bits.



Registros de segmento de 16 bits



Nota: La figura describe los registros de propósitos generales para una arquitectura de 32 bits. Fuente: Irvine (2008).

Figura 2

Registros de propósito general de 64 bits.

Ingeniería en Desarrollo
de Software



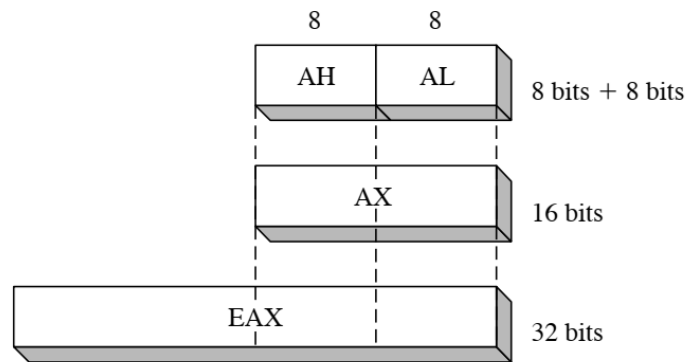
Nota: Registros de propósito general de un procesador x86-64. Fuente: Acevedo (2019)

Para Irvine (2018) el propósito de cada registro es descrito de la siguiente manera:

Registros de propósito general: Los registros de propósito general se utilizan principalmente para las operaciones aritméticas y el movimiento de datos. Como se muestra en la siguiente figura 3, cada registro puede direccionarse como un valor individual de 32 bits, o como dos valores de 16 bits.

Figura 3

Direccionamiento de registros.



Nota: La figura describe la forma en que los registros de 32 bits se pueden direccionar en registros más pequeños. Fuente: Irvine (2008)

Hay partes de algunos registros que pueden direccionarse como valores de 8 bits. Por ejemplo, el registro EAX de 32 bits tiene una mitad inferior de 16 bits llamada AX. A su vez, el registro AX tiene una mitad superior de 8 bits llamada AH, y una mitad inferior de 8 bits llamada AL. La misma relación de traslape existe para los registros EAX, EBX, ECX y EDX:

Tabla 2

División de registros.

32 bits	16 bits	8 bits (superior)	8 bits (inferior)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL

Nota: La figura describe la forma en que los registros de 32 bits se pueden dividir en registros más pequeños. Fuente: Irvine (2008)

Registros de segmento: En el modo de direccionamiento real, los registros de segmento, indican las direcciones base de las áreas preasignadas de memoria, conocidas como segmentos. En el modo protegido, los registros de segmento guardan apuntadores a tablas de descriptores de segmento. Algunos segmentos guardan instrucciones de un programa (código), otros

guardan variables (datos), y otro segmento llamado segmento de pila guarda las variables de funciones locales y los parámetros de funciones.

Registro EFLAGS: El registro EFLAGS (o simplemente Flags) consiste en bits binarios individuales que controlan la operación de la CPU, o que reflejan el resultado de alguna operación de la CPU. Algunas instrucciones evalúan y manipulan las banderas individuales del procesador.

Banderas de control: Las banderas de control controlan la operación de la CPU. Por ejemplo, pueden hacer que la CPU salga de un ciclo después de ejecutar cada instrucción, generar una interrupción cuando se detecta un desbordamiento aritmético, entrar al modo 8086 virtual y entrar al modo protegido. Los programas pueden activar bits individuales en el registro EFLAGS para controlar la operación de la CPU. Algunos ejemplos son las banderas de Dirección y de Interrupción.

Banderas de estado: Las banderas de estado reflejan los resultados de las operaciones aritméticas y lógicas que realiza la CPU. Estas banderas son: Desbordamiento, Signo, Cero, Acarreo Auxiliar, Paridad y Acarreo.

Apuntador de instrucciones: El registro EIP, o apuntador de instrucciones, contiene la dirección de la siguiente instrucción a ejecutar. Ciertas instrucciones de máquina manipulan a EIP, para que el programa se bifurque hacia una nueva ubicación.

2.4 Administración de memoria del procesador IA-32 Modo de direccionamiento Real y Protegido.

Los procesadores IA-32 administran la memoria de acuerdo a los modos básicos de operación que se estudiaron anteriormente:

- Modo Real

- Modo protegido
- Modo 8086 virtual.

Irvine (2008) los detalla de la siguiente manera:

En el modo de direccionamiento real sólo puede direccionarse 1MB de memoria, del 00000 al FFFFF hexadecimal. El procesador sólo puede ejecutar un programa a la vez, pero puede interrumpir en forma momentánea ese programa para procesar las solicitudes (conocidas como interrupciones) de los periféricos. Los programas de aplicación pueden leer y modificar cualquier área de la RAM (memoria de acceso aleatorio) y pueden leer, pero no modificar cualquier área de la ROM (memoria de sólo lectura). El sistema operativo MS-DOS se ejecuta en modo de direccionamiento real, y Windows 95/98 puede cargarse en este modo.

En el modo protegido, el procesador puede ejecutar varios programas al mismo tiempo. A cada proceso (programa en ejecución) le asigna un total de 4GB de memoria. A cada programa se le puede asignar su propia área reservada de memoria, y los programas no pueden acceder de manera accidental al código y los datos de los demás programas. MS-Windows y Linux se ejecutan en modo protegido.

En el modo 8086 virtual, la computadora se ejecuta en modo protegido y crea una máquina 8086 virtual con su propio espacio de direcciones de 1MB, que simula a una computadora 80 × 86 que se ejecuta en modo de direccionamiento real. Por ejemplo, Windows NT y 2000 crean una máquina 8086 virtual cuando abrimos una ventana de Comandos. Puede ejecutar muchas de esas ventanas al mismo tiempo, y cada una está protegida contra las acciones de las demás. Algunos programas de MS-DOS que hacen referencias directas al hardware de la computadora no se ejecutarán en este modo bajo Windows NT, 2000 y XP.

NOTA: Para más detalle leer (Lenguaje ensamblador para computadoras basadas en Intel., 2008, págs. 39-43)

Semana 7

2.5 Unidad Aritmética y Lógica – ALU

Stallings (2005) describe la ALU de la siguiente manera:

La ALU es la parte del computador que realiza realmente las operaciones aritméticas y lógicas con los datos. El resto de los elementos del computador (unidad de control, registros, memoria, E/S) están principalmente para suministrar datos a la ALU, a fin de que esta los procese y para recuperar los resultados. Con la ALU llegamos al estudio de lo que puede considerarse el núcleo o esencia del computador. (p. 302)

Orenga & Manonellas (2011) dentro del módulo El procesador aportan el siguiente concepto:

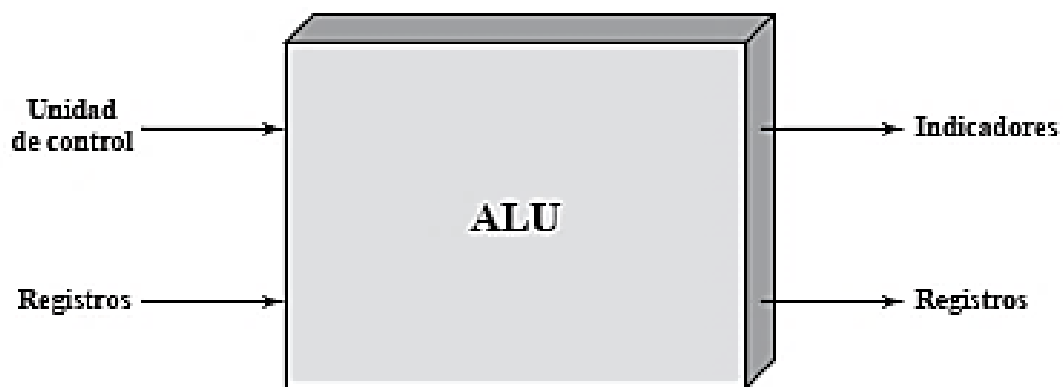
La unidad aritmética y lógica o ALU es un circuito combinacional capaz de realizar operaciones aritméticas y lógicas con números enteros y también con números reales. Las operaciones que puede efectuar vienen definidas por el conjunto de instrucciones aritméticas y lógicas de las que dispone el juego de instrucciones del computador. (p. 17)

De manera sencilla podemos decir que la ALU es la encargada de realizar todas las operaciones aritméticas (suma, resta, multiplicación, división) y operaciones lógicas que incluyen AND, OR, NOT XOR, operaciones de incremento positivo (+1) o negativo (-1).

En la figura 4 se indica en términos generales como se interconecta la ALU con el resto del procesador.

Figura 4

Entradas y salidas de la ALU



Nota: La figura describe los parámetros de entrada y salida que se proporcionan a la ALU.
Fuente: Stallings (2005)

Los datos se presentan a la ALU en registros y en registros se almacenan los resultados de las operaciones producidos por la ALU. Estos registros son posiciones de memoria temporal internas al procesador que están conectados a la ALU. La ALU puede también activar indicadores (flags) como resultado de una operación.

2.6 Unidad de Control

Orenga & Manonellas (2011) en el módulo El procesador describen la Unidad de Control de la siguiente manera:

La unidad de control se puede considerar el cerebro del computador. Como el cerebro, está conectada al resto de los componentes del computador mediante las señales de control (el sistema nervioso del computador). Con este símil no se pretende humanizar los computadores, sino ilustrar que la unidad de control es imprescindible para coordinar los diferentes elementos que tiene el computador y hacer un buen uso de ellos.

La función básica de la unidad de control es la ejecución de las instrucciones, pero su complejidad del diseño no se debe a la complejidad de estas tareas (que en general son muy sencillas), sino a la sincronización que se debe hacer de ellas. (p. 20)

La unidad de control, proporciona señales que activan las microoperaciones de la ruta de datos para llevar a cabo tareas concretas. La unidad de control también determina la secuencia en la que se realizan las diversas operaciones. El diseño de un sistema digital se trata, en general, dividiéndolo en dos partes bien diferenciadas, los registros y sus transferencias.

Generalmente, la temporización de todos los registros en un sistema digital síncrono se controla mediante un reloj maestro. Los pulsos de reloj se aplican a todos los flip-flops y registros del sistema, incluyendo los de la unidad de control. Para evitar que cambie el contenido de los registros en cada pulso de reloj, algunos de ellos llevan una señal de control de carga que habilita o deshabilita la carga de un nuevo dato en dichos registros. Las variables binarias que controlan las entradas de selección de los multiplexores, los buses, la lógica de procesamiento y las entradas de control de carga de los registros se generan, por lo general, en la unidad de control.

Semana 8

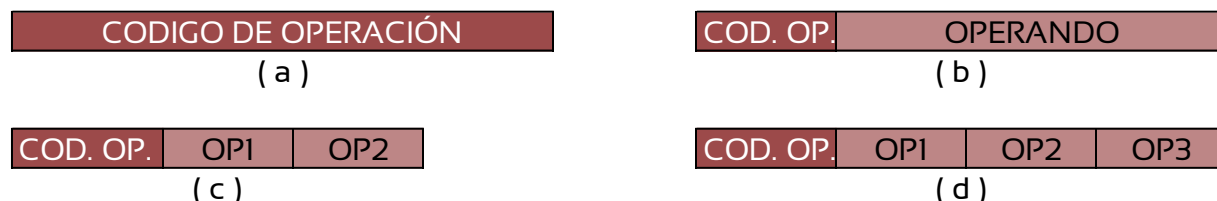
Ingeniería en Desarrollo de Software

2.7 Arquitectura y representación del Juego de Instrucciones.

Una instrucción es un código de operación el cual indicará que proceso debe realizar la computadora, este código de operación es acompañado por **operandos**, de los cuales se pueden incluir cero, uno dos o tres de ellos, estos pueden almacenar tanto el origen como el destino de los datos que están siendo operados u otros valores. De esta manera podemos decir que cada instrucción incluye toda la información necesaria para ser ejecutada.

Figura 5

Formatos de instrucciones comunes.



Cuatro formatos de instrucciones comunes (a) Instrucción con cero operandos. (b) Instrucción con un operando. (c) Instrucción con dos operandos. (d) Instrucción con tres operandos.

Nota: La figura describe los formatos comunes de las instrucciones. Fuente: Tanenbaum (2000).

Orenga & Manonellas (2011) en el módulo Juego de instrucciones destaca lo siguiente:

Uno de los elementos necesarios en cualquier instrucción es el **conjunto de operandos**. Los operandos necesarios para ejecutar una instrucción pueden encontrarse explícitamente en la instrucción o pueden ser implícitos.

La localización dentro del procesador de los operandos necesarios para ejecutar una instrucción y la manera de explicitarlos dan lugar a arquitecturas diferentes del juego de instrucciones. (p.8)

Las arquitecturas que podemos destacar según Orenga & Manonellas (2011) son las siguientes:

- **Arquitecturas basadas en pila:** los operandos son implícitos y se encuentran en la pila.
- **Arquitecturas basadas en acumulador:** uno de los operandos se encuentra de manera implícita en un registro denominado acumulador.
- **Arquitecturas basadas en registros de propósito general:** los operandos se encuentran siempre de manera explícita, ya sea en registros de propósito general o en la memoria.

Dentro de las arquitecturas basadas en registros de propósito general, podemos distinguir tres subtipos:

- **Registro-registro (o load-store).** Solo pueden acceder a la memoria instrucciones de carga (load) y almacenamiento (store).
- **Registro-memoria.** Cualquier instrucción puede acceder a la memoria con uno de sus operandos.
- **Memoria-memoria.** Cualquier instrucción puede acceder a la memoria con todos sus operandos.

Podemos hablar de cinco tipos de arquitectura del juego de instrucciones:

- 1) Pila
- 2) Acumulador
- 3) Registro-registro
- 4) Registro-memoria
- 5) Memoria-memoria

En la tabla 3 se muestra gráficamente la diferencia entre los tipos de arquitectura, veremos como se resuelve la misma operación: sumar dos valores (A y B) almacenados en memoria y guardar el resultado en una tercera posición de memoria (C).

2.7.2 Representación del juego de instrucciones.

Orenga & Manonellas (2011) describe que el juego de instrucciones se representa desde dos puntos de vista:

- 1) Desde el punto de vista del **computador**, cada instrucción se representa como una secuencia de bits que se divide en campos, en los que cada campo corresponde a uno de los elementos que forman la instrucción. Cada bit es la representación de una señal eléctrica alta o baja. Esta manera de representar

el juego de instrucciones se suele denominar *código de máquina* o *lenguaje de máquina*.

Tabla 3

Arquitecturas del juego de instrucciones.

Pila	Acumulador	Registro-registro	Registro-memoria	Memoria-memoria
PUSH [A]	LOAD [A]	LOAD R1 [A]	LOAD R1 [A]	MOV [C] [A]
PUSH [B]	ADD [B]	LOAD R2, [B]	ADD R1, [B]	ADD [C], [B]
ADD	STORE [C]	ADD R2, R1	MOV [C], R2	
POP [C]		STORE [C], R2		

Hemos supuesto que en las instrucciones de dos operandos, el primer operando actúa como operando fuente y operando destino en el ADD y solo como operando destino en el resto, el segundo operando actúa siempre como operando fuente. A, B y C son las etiquetas que representan las direcciones de memoria donde están almacenados los valores que queremos sumar y su resultado, respectivamente. Los corchetes indican que tomamos el contenido de estas direcciones de memoria.

Nota: La tabla describe las diferencias entre los tipos de arquitectura. Fuente: Orenge & Manonellas (2011).

- Desde el punto de vista del **programador**, las instrucciones se representan mediante símbolos y se utilizan expresiones mnemotécnicas o abreviaturas. Hay que tener presente que es muy difícil trabajar con las representaciones binarias propias del código de máquina. Esta manera de representar el juego de instrucciones se suele denominar *código de ensamblador* o *lenguaje de ensamblador*.

Tabla 4

Código ensamblador desde el punto de vista del programador.

Código de operación	Operando 1	Operando 2
ADD	R1	R2

Nota: La tabla describe código ensamblador desde el punto de vista del programador. Fuente: Orenge & Manonellas (2011).

Tabla 5

Código maquina desde el punto de vista del computador.

Código de operación	Operando 1	Operando 2
20h	11h	12h
0010 0000	0001 0001	0001 0010
8 bits	8 bits	8 bits
← 24 bits →		

Nota: La tabla describe código maquina desde el punto de vista del computador. Fuente: Orenge & Manonellas (2011).

2.8 Formato de Instrucciones.

2.8.1 Elementos que componen una instrucción.

Orenge & Manonellas (2011) describe los elementos que componen una instrucción:

- **Código de operación:** especifica la operación que hace la instrucción.
- **Operando fuente:** para hacer la operación pueden ser necesarios uno o más operandos fuente; uno o más de estos operandos pueden ser implícitos.
- **Operando destino:** almacena el resultado de la operación realizada. Puede estar explícito o implícito. Uno de los operandos fuente se puede utilizar también como operando destino.
- **Dirección de la instrucción siguiente:** especifica dónde está la instrucción siguiente que se debe ejecutar; suele ser una información implícita, ya que el procesador va a buscar automáticamente la instrucción que se encuentra a continuación de la última instrucción ejecutada. Solo las instrucciones de ruptura de secuencia especifican una dirección alternativa.

2.8.2 Tamaño de las instrucciones

En el formato de las instrucciones uno de los aspectos importantes a la hora de diseñar las instrucciones es determinar su tamaño. En este sentido Orenge & Manonellas (2011) muestran dos alternativas:

- **Instrucciones de tamaño fijo:** todas las instrucciones ocuparán el mismo número de bits. Esta alternativa simplifica el diseño del procesador y la ejecución de las instrucciones puede ser más rápida.
- **Instrucciones de tamaño variable:** el tamaño de las instrucciones dependerá del número de bits necesario para cada una. Esta alternativa permite diseñar un conjunto amplio de códigos de operación, el direccionamiento puede ser más flexible y permite poner referencias a registros y memoria. Como contrapartida, aumenta la complejidad del procesador.

2.8.3 Tipos de instrucciones.

Los códigos de operación de las instrucciones pueden ser diferente de una máquina a otra, pero podemos encontrar los mismos tipos de instrucciones las cuales se pueden clasificar de una manera común, Orenge & Manonellas (2011) nos muestran su clasificación:

- Transferencia de datos
- Aritméticas
- Lógicas
- Transferencia del control
- Entrada/salida
- Otros tipos

Nota: Para más detalle puede consultar el libro (Estructura de computadores, 2011) el módulo de Juego de Instrucciones.

Semana 9

2.9 Fundamentos del Lenguaje Ensamblador en Sistemas GNU/Linux

2.9.1 Elementos básicos del lenguaje ensamblador.

Antes de comenzar a programar en lenguaje ensamblador es necesario conocer cada una de las herramientas que posee este lenguaje y la forma en que se pueden utilizar. Irvine (2008) nos describe los elementos que debemos conocer previo a iniciar la programación de programas en ensamblador:

1) Constantes enteras

Esta compuesta de un signo opcional a la izquierda, uno o más dígitos y un carácter opcional de sufijo (llamado raíz) que indica la base numérica:

[{ + | - }] dígitos [raíz]

La raíz puede ser una de las siguientes:

- Hexadecimal h
- Octal q/o
- Decimal d
- Binario b

Algunos ejemplos serían

25	Decimal
25d	Decimal
1110011b	Binario
50o	Octal
0AFh	Hexadecimal

2) Expresiones enteras

Una expresión entera es una expresión matemática que involucra valores enteros y operadores aritméticos. La Tabla 6 presenta los operadores aritméticos de acuerdo a su orden de precedencia:

Tabla 6

Operadores aritméticos.

Operador	Nombre	Nivel de precedencia
()	Paréntesis	1
+, -	Unario positivo, Unario negativo	2
*, /	Multiplicación, división	3
MOD	Módulo	4
+, -	Suma, resta	5

Nota: La tabla describe los operadores aritméticos y su nivel de precedencia. Fuente: Irvine (2008).

3) Constantes tipo carácter

Una constante tipo carácter es un solo carácter encerrado entre comillas sencillas o dobles. Ejemplo:

'A'

"d"

4) Constantes tipo cadena

Una constante tipo cadena es una secuencia de caracteres (inclusive espacios) encerrados entre comillas sencillas o dobles:

'ABC'

"Hola Mundo"

5) Palabras reservadas

Las palabras reservadas tienen un significado especial en MASM, y sólo pueden usarse dentro de su contexto correcto. Hay distintos tipos de palabras reservadas:

- Nemónicos de instrucciones, como MOV, ADD y MUL.
- Directivas, las cuales indican cómo ensamblar programas.
- Atributos, que proporcionan información acerca del tamaño y uso de las variables y operandos. Dos ejemplos son BYTE y WORD.
- Operadores, que se utilizan en expresiones constantes.
- Símbolos predefinidos como @data, que devuelven valores enteros constantes en tiempo de ensamblado.

6) Identificadores

Un identificador es un nombre elegido por el programador. Puede servir para identificar a una variable, una constante, un procedimiento o una etiqueta de código.

Considere lo siguiente a la hora de crear identificadores:

- Pueden contener entre 1 y 247 caracteres.
- No son sensibles a mayúsculas/minúsculas.
- El primer carácter debe ser una letra (A..Z, a..z), guion bajo (_), @, ? o \$. Los caracteres subsiguientes también pueden ser dígitos.
- Un identificador no puede ser igual que una palabra reservada para el lenguaje ensamblador.

7) Directivas

Una directiva es un comando incrustado en el código fuente, que el ensamblador reconoce y actúa en base a ésta. Las directivas no se llevan a cabo en tiempo de ejecución, mientras que las instrucciones sí. Las directivas pueden definir variables, macros y procedimientos. Pueden asignar nombres a los segmentos de memoria y realizar muchas otras tareas de mantenimiento relacionadas con el ensamblador.

8) Instrucciones

Una instrucción es un enunciado que se vuelve ejecutable cuando se ensambla un programa. El ensamblador traduce las instrucciones en bytes de lenguaje máquina, para que la CPU los cargue y los lleve a cabo en tiempo de ejecución. Una instrucción contiene cuatro partes básicas:

- Etiqueta (opcional).

- Nemónico de instrucción (requerido).
- Operando(s) (por lo general, son requeridos).
- Comentario (opcional).

Ésta es la sintaxis básica:

[etiqueta:] nemónico operando(s) [; comentario]

NOTA: Para mas detalles puedes consultar (Lenguaje ensamblador para computadoras basadas en Intel., 2008) Capitulo 3.

Semana 10

2.9 Fundamentos del Lenguaje Ensamblador en Sistemas GNU/Linux

Antes de iniciar debes de tomar en cuenta que para programar en lenguaje ensamblador tenemos diferentes sintaxis incluyendo TASM (Turbo Assembler), NASM (Netwide Assembler) y el ensamblador para Linux GAS (GNU Assembler).

En esta sección utilizaremos NASM (Netwide Assembler) ya que sus sintaxis esta basada en la sintaxis Intel. El entorno de ejecución será en el entorno Linux de su preferencia (Debian, Ubuntu, Linux Mint, etc.)

2.9.1 Estructura de un programa en ensamblador.

Orenga & Manonellas (2011) nos muestra las secciones que podemos encontrar en un programa ensamblador:

Un programa ensamblador escrito con sintaxis NASM está formado por tres secciones o segmentos: .data para los datos inicializados, .bss para los datos no inicializados y .text para el código:

- section .data
- section .bss
- section .text

Para definir una sección se pueden utilizar indistintamente las directivas `section` y `segment`.

La sección `.bss` no es necesaria si se inicializan todas las variables que se declaran en la sección `.data`.

La sección `.text` permite también definir variables y, por lo tanto, permitiría prescindir también de la sección `.data`, aunque no es recomendable. Por lo tanto, esta es la única sección realmente obligatoria en todo programa.

La sección `.text` debe empezar siempre con la directiva global, que indica al GCC cuál es el punto de inicio del código. Cuando se utiliza GCC para generar un ejecutable, el nombre de la etiqueta donde se inicia la ejecución del código se ha de denominar obligatoriamente `main`; también ha de incluir una llamada al sistema operativo para finalizar la ejecución del programa y devolver el control al terminal desde el que hemos llamado al programa.

En la figura 6 se muestra un ejemplo de programa en ensamblador:

Figura 6

Ejemplo programa en ensamblador.

Ingeniería en Desarrollo
de Software


```

;1: fichero hola.asm
section .data ;2: Inicio de la sección de datos
    msg db "Hola!",10 ;3:
;4: El 10 corresponde al código ASCII del salto de línea.
;5:
section .text ;6: Inicio de la sección de código.
global main ;7: Esta directiva es para hacer visible
;8: una etiqueta para el compilador de C.
;9:
main: ;10: Por defecto el compilador de C reconoce como
;11: punto de inicio del programa la etiqueta main.
;12: Mostrar un mensaje
mov rax,4 ;13: Pone el valor 4 en el registro rax
;14: para hacer la llamada a la función write (sys_write)
mov rbx,1 ;15: Pone el valor 1 en el registro RBX
;16: para indicar el descriptor que hace referencia
;17: a la salida estándar.
mov rcx,msg ;18: Pone la dirección de la variable msg
;19: en el registro RCX
mov rdx,6 ;20: Pone la longitud del mensaje incluido el 10
;21: del final en el registro RDX
int 80h ;22: llama al sistema operativo
;23:
;24: devuelve el control al terminal del sistema operativo.
mov rax,1 ;25: Pone el valor 1 en el registro rax
;26: para hacer la llamada a la función exit (sys_exit)
mov rbx,0 ;27: Pone el valor 0 en el registro RBX
;28: para indicar el código de retorno (0=sin errores)
int 80h ;29: llama al sistema operativo
;30:

```

Nota: La figura describe el código de un programa en ensamblador básico. Fuente: Orenga & Manonellas (2011).

Nota: En el ejemplo previo se usan registros de 64 bits, pero se pueden sustituir con registros de 32 bits.

2.9.2 Directivas

En el módulo de Programación en ensamblador Orenga & Manonellas (2011) describe las directivas como:

Pseudooperaciones que solo son reconocidas por el ensamblador. No se deben confundir con las instrucciones, a pesar de que en algunos casos pueden añadir código a nuestro programa. Su función principal es declarar ciertos elementos de nuestro programa para que puedan ser identificados más fácilmente por el programador y también para facilitar la tarea de ensamblaje.

- **Definición de constantes**

Una constante es un valor que no puede ser modificado por ninguna instrucción del código del programa.

Para definir constantes se utiliza la directiva equ, de la manera siguiente:
nombre_constante equ valor

Ejemplo:

```
tamañoVec equ 5
ServicioSO equ 80h
Mensaje equ 'Hola'
```

- **Definición de variables**

La declaración de variables en un programa en ensamblador se puede incluir en la sección .data o en la sección .bss, según el uso de cada una.

Sección .data , variables inicializadas

Las variables de esta sección se definen utilizando las siguientes directivas:

- db: define una variable de tipo byte, 8 bits.
- dw: define una variable de tipo palabra (word), 2 bytes = 16 bits.
- dd: define una variable de tipo doble palabra (double word), 2 palabras = 4 bytes = 32 bits.
- dq: define una variable de tipo cuádruple palabra (quad word), 4 palabras = 8 bytes = 64 bits.

Ejemplos

```
var1 db 255 ; define una variable con el valor FFh
Var2 dw 65535 ; en hexadecimal FFFFh
var4 dd 4294967295 ; en hexadecimal FFFFFFFFh
var8 dq 18446744073709551615 ; en hexadecimal
FFFFFFFFFFFFFFFFh
```

Con la misma sintaxis se pueden declarar variables de todos los tipos de datos vistos anteriormente:

- Carácter
- Cadenas de caracteres
- Números (Decimal, Octal, Binario, Hexadecimal)

Sección .bss, variables no inicializadas.

Dentro de esta sección se declaran y se reserva espacio para las variables de nuestro programa para las cuales no queremos dar un valor inicial.

Hay que utilizar las directivas siguientes para declarar variables no inicializadas:

- `resb`: reserva espacio en unidades de byte
- `resw`: reserva espacio en unidades de palabra, 2 bytes
- `resd`: reserva espacio en unidades de doble palabra, 4 bytes
- `resq`: reserva espacio en unidades de cuádruple palabra, 8 bytes

Ejemplo:

```
section .bss

var1 resb 1 ;reserva 1 byte
var2 resb 4 ;reserva 4 bytes
var3 resw 2 ;reserva 2 palabras = 4 bytes, equivalente al caso anterior
var3 resd 1 ;reserva una cuádruple palabra = 4 bytes
           ;equivalente a los dos casos anteriores
```

2.9.3 Tipos de Instrucciones

El juego de instrucciones de los procesadores x86-32 es muy amplio. Podemos organizar las instrucciones según los tipos siguientes:

1) Instrucciones de transferencia de datos:

- **mov destino, fuente**: instrucción genérica para mover un dato desde un origen a un destino.

- **push fuente:** instrucción que mueve el operando de la instrucción a la cima de la pila.
- **pop destino:** mueve el dato que se encuentra en la cima de la pila al operando destino.
- **xchg destino, fuente:** intercambia contenidos de los operandos.

2) Instrucciones aritméticas y de comparación:

- **add destino, fuente:** suma aritmética de los dos operandos.
- **adc destino, fuente:** suma aritmética de los dos operandos considerando el bit de transporte.
- **sub destino, fuente:** resta aritmética de los dos operandos.
- **sbb destino, fuente:** resta aritmética de los dos operandos considerando el bit de transporte.
- **inc destino:** incrementa el operando en una unidad.
- **dec destino:** decrementa el operando en una unidad.
- **mul fuente:** multiplicación entera sin signo.
- **imul fuente:** multiplicación entera con signo.
- **div fuente:** división entera sin signo.
- **idiv fuente:** división entera con signo.
- **neg destino:** negación aritmética en complemento a 2.
- **cmp destino, fuente:** comparación de los dos operandos; hace una resta sin guardar el resultado.

3) Instrucciones lógicas y de desplazamiento:

a) Operaciones lógicas:

- **and destino, fuente:** operación 'y' lógica.
- **or destino, fuente:** operación 'o' lógica.
- **xor destino, fuente:** operación 'o exclusiva' lógica.
- **not destino:** negación lógica bit a bit.

- **test destino, fuente:** comparación lógica de los dos operandos; hace una 'y' lógica sin guardar el resultado.

b) Operaciones de desplazamiento:

- **sal destino, fuente / shl destino, fuente:** desplazamiento aritmético/lógico a la izquierda.
- **sar destino, fuente:** desplazamiento aritmético a la derecha.
- **Shr destino, fuente:** desplazamiento lógico a la derecha.
- **rol destino, fuente:** rotación lógica a la izquierda.
- **ror destino, fuente:** rotación lógica a la derecha.
- **rcl destino, fuente:** rotación lógica a la izquierda considerando el bit de transporte.
- **rcr destino, fuente:** rotación lógica a la derecha considerando el bit de transporte.

4) Instrucciones de ruptura de secuencia:

a) Salto incondicional:

- **jmp etiqueta:** salta de manera incondicional a la etiqueta.

Semana 11

Ingeniería en Desarrollo de Software

2.10 Uso del Lenguaje Ensamblador con C en Sistemas GNU/Linux

2.10.1 Conceptos de programación en ensamblador y C

Acceso a datos

Para acceder a datos de memoria en ensamblador, como sucede en los lenguajes de alto nivel, lo haremos por medio de variables que deberemos definir previamente para reservar el espacio necesario para almacenar la información. En la definición podemos especificar el nombre de la variable, el tamaño y un valor inicial; para

acceder al dato lo haremos con los operandos de las instrucciones especificando un modo de direccionamiento determinado.

Por ejemplo, `var dd 12345678h` es la variable con el nombre `var` de tamaño 4 bytes inicializada con el valor `12345678h`. `mov eax, dword[var]` es la instrucción en la que accedemos a la variable `var` utilizando direccionamiento directo a memoria, en la que especificamos que queremos acceder a 4 bytes (`dword`) y transferimos el contenido, el valor `12345678h`, al registro `eax`.

En ensamblador hay que estar muy alerta cuando accedemos a las variables que hemos definido. Las variables se guardan en memoria consecutivamente a medida que las declaramos y no existe nada que delimite las unas de las otras.

Ejemplo

```
.data
var1 db 0           ;variable definida de 1 byte
var2 db 61h         ;variable definida de 1 byte
var3 dw 0200h       ;variable definida de 2 bytes
var4 dd 0001E26Ch   ;variable definida de 4 bytes
```

Las variables se encontrarán en memoria tal como muestra la tabla 7.

Tabla 7

Variables en memoria

Dirección	Valor
var1	00h
var2	61h
var3	00h
	02h
var4	6Ch
	E2h
	01h
	00h

Nota: La tabla describe las variables y sus valores en memoria. Fuente: Orenga & Manonellas (2011).

Si ejecutamos la instrucción siguiente:

```
mov eax, dword[var1]
```

Cuando accedemos a `var1` como una variable de tipo `DWORD` el procesador tomará como primer byte el valor de `var1`, pero también los 3 bytes que están a continuación, por lo tanto, como los datos se tratan en formato little-endian, consideraremos `DWORD [var1] = 02006100h` y este es el valor que llevaremos a `EAX` (`eax=02006100h`). Si este acceso no es el deseado, el compilador no reportará ningún error, ni tampoco se producirá un error durante la ejecución; solo podremos detectar que lo estamos haciendo mal probando el programa y depurando.

Por una parte, el acceso a los datos es muy flexible, pero, por otra parte, si no controlamos muy bien el acceso a las variables, esta flexibilidad nos puede causar ciertos problemas. En lenguaje C tenemos un comportamiento parecido. Supongamos que tenemos el código siguiente:

```
int vec[4]={1,2,3,4};
int x=258; //258=00000102h
char c=3;

int main() {
    int i=0;
    for (i=0;i<5;i++) printf("contenido de vec", vec[i]);
    c = x;
}
```

El índice `i` toma los valores 0, 1, 2, 3 y 4, el índice `i=4` no corresponde a una posición del vector; la primera posición es la 0 y la última es la 3; por lo tanto, estamos accediendo a una posición fuera del vector. Al compilar el código, no se generará ningún error y tampoco se producirá ningún error durante la ejecución. Si la variable `x` ocupara la posición de memoria siguiente a continuación de los 4 elementos que tenemos definidos del vector, se mostraría un 258.

En la asignación `c = x` como los tipos son diferentes, `x` (int de 4 bytes) y `c` (char de 1 byte), asigna el byte menos significativo de `x` a la variable `c`, después de la asignación `c = 2`. Si hiciéramos la asignación al revés, `x = c`, haría la extensión de signo de `c` a 4 bytes y, por lo tanto, después de la asignación `x = 3` (00000003h).

A diferencia del ensamblador, en el que las variables se encuentran en la memoria en el mismo orden en el que las declaramos, en C no podemos asegurarlo, ya que la reserva de espacio de memoria para las variables que hace el compilador GCC es diferente.

Estructuras de datos

Veamos cómo acceder a vectores y matrices utilizando lenguaje C y lenguaje de ensamblador. Definimos en C un vector y una matriz y hacemos la suma de los elementos.

```
int main() {  
  
    int vec[6]={1,2,3,4,5,6},mat[2][3]={{1,2,3},{4,5,6}};  
    int i,j, sumaVec=0, sumaMat=0;  
  
    for (i=0;i<6;i++) sumaVec=sumaVec+vec[i];  
  
    for (i=0;i<2;i++){  
        for (j=0;j<3;j++) sumaMat=sumaMat+mat[i][j];  
    }  
}
```

A continuación, veremos cómo hacer lo mismo con lenguaje de ensamblador, utilizando diferentes variantes del direccionamiento indexado (el formato general es [dirección + Registro Base + Registro Índice * escala]):


```

vec dd 1,2,3,4,5,6
mat dd 1,2,3
    dd 4,5,6

;recorremos el vector para sumarlo
mov esi,0 ;esi será el índice para acceder a los datos
mov eax,0 ;eax será donde guardaremos la suma
loop_vec:
    add eax, dword[vec+esi*4] ;multiplican el índice por 4
    inc esi ;porque cada elemento ocupa 4 bytes.
    cmp esi, 6 ;comparamos con 6 porque es el índice del primer
    ;elemento fuera del vector.
    jl loop_vec

;recorremos la matriz con un solo índice para sumarla
mov esi,0 ;esi será el índice para acceder a los datos
mov ebx,0 ;ebx será donde guardaremos la suma
loop_mat:
    add ebx, dword[mat+esi*4] ;multiplican el índice por 4
    inc esi ;porque cada elemento ocupa 4 bytes.
    cmp esi, 6 ;comparamos con 6 porque es el índice del primer elemento
    ;fuera de la matriz, la matriz tiene 2*3=6 elementos.
    jl loop_mat

;recorremos la matriz con dos índices para sumarla
mov esi,0 ;esi será el índice de la columna
mov edi,0 ;edi será el índice de la fila
mov ebx,0 ;ebx será donde guardaremos la suma

loop_mat2:
    add ebx, dword[mat+edi+esi*4] ;multiplican el índice de la columna
    inc esi ;por 4 porque cada elemento ocupa 4 bytes.
    cmp esi, 3 ;comparamos con 3 porque son los elementos de una fila.
    jl loop_mat2
    mov esi, 0
    add edi, 12 ;cada fila ocupa 12 bytes, 3 elementos de 4 bytes.
    cmp edi, 24 ;comparamos con 24 porque es el primer índice
    ;fuera de la matriz.
    jl loop_mat2

```

Como podemos comprobar, el código para acceder al vector y a la matriz utilizando un índice son idénticos; en ensamblador las matrices se ven como vectores y todas las posiciones de memoria son consecutivas; por lo tanto, si queremos acceder a una posición concreta de una matriz, a partir de un número de fila y de columna,

deberemos calcular a qué posición de memoria hay que acceder; $\text{fila} \times \text{elementos_de_la_fila} \times \text{tamaño_del_dato} + \text{columna} \times \text{tamaño_del_dato}$.

Por ejemplo, si queremos acceder al elemento `mat[1][2]`, segunda fila, tercera columna (las filas y columnas se empiezan a numerar por 0), este elemento estará en la posición $1 \times 3 \times 4 + 2 \times 4 = 20$, `[mat+20]`.

Operaciones aritméticas

En este subapartado hemos de tener presente que en C podemos construir expresiones utilizando variables, constantes y operadores en una misma sentencia; en cambio, hacer lo mismo en ensamblador implica escribir una secuencia de instrucciones en la que habrá que hacer las operaciones una a una según la prioridad de los operadores dentro de la sentencia.

Ejemplo

Sentencia que podemos expresar en lenguaje C:

```
r=(a+b)*4 / (c>>2);
```

Traducción de la sentencia en lenguaje de ensamblador:

```
mov eax, [a]
mov ebx, [b]
add eax, ebx ; (a+b)
imul eax, 4 ; (a+b)*4
mov ecx, [c]
sar ecx, 2 ; (c>>2)
idiv ecx ; (a+b)*4 / (c>>2)
mov [r], eax ; r=(a+b)*4 / (c>>2)
```

Control de flujo

En este subapartado veremos cómo las diferentes estructuras de control del lenguaje C se pueden traducir a lenguaje de ensamblador.

Estructura if

Estructura condicional expresada en lenguaje C:

```
if (condición) {  
    bloque de sentencias  
}
```

Ejemplo

```
if (a > b) {  
    maxA = 1;  
    maxB = 0;  
}
```

Se puede traducir a lenguaje de ensamblador de la manera siguiente:

```
mov rax, qword [a]      ;Se cargan las variables en registros  
mov rbx, qword [b]  
  
cmp rax, rbx            ;Se hace la comparación  
jg cierto               ;Si se cumple la condición, salta a la etiqueta cierto  
jmp fin                 ;Si no se cumple la condición, salta a la etiqueta fin  
  
cierto:  
    mov byte [maxA], 1 ;Estas instrucciones solo se ejecutan  
    mov byte [maxB], 0 ;cuando se cumple la condición  
  
fin:
```

Estructura while

Estructura iterativa controlada por una condición expresada al principio:

```
while (condiciones) {  
    bloque de sentencias que ejecutar  
}
```

Ejemplo

```
resultado=1;  
while (num > 1){ //mientras num sea mayor que 1 hacer ...  
    resultado = resultado * num;  
    num--;  
}
```

Se puede traducir a lenguaje de ensamblador de esta manera:

```

mov rax, 1          ;rax será [resultado]
mov rbx, qword [num] ;Se carga la variable en un registro
while:
  cmp rbx, 1          ;Se hace la comparación
  jg cierto           ;Si se cumple la condición (num > 1) salta a cierto
  jmp fin             ;Si no, salta a fin
cierto:
  imul rax, rbx        ;rax = rax * rbx
  dec rbx
  jmp while
fin:
  mov qword [resultado], rax
  mov qword [num], rbx

```

Estructura for

Estructura iterativa, que utiliza la orden for:

```

for (valores iniciales; condiciones; actualización) {
    bloque de sentencias que ejecutar
}

```

Ejemplo

```

resultado=1;
for (i = num; i > 1; i--)
{
    resultado=resultado*i;
}

```

Se puede traducir a lenguaje de ensamblador de esta manera:

```

mov rax, 1          ;rax será [resultado]
mov rcx, qword [num] ;rcx será [i] que inicializamos con [num]

for:
  cmp rcx, 1          ;Se hace la comparación
  jg cierto           ;Si se cumple la condición, salta a cierto
  jmp fin             ;Si no, salta a fin
cierto:
  imul rax, rcx
  dec rcx
  jmp for
fin:
  mov qword [resultado], rax
  mov qword [i], rcx

```

Al salir de la estructura iterativa, no actualizamos la variable num porque se utiliza para inicializar i pero no se cambia.

Semana 12

2.10 Uso del Lenguaje Ensamblador con C en Sistemas GNU/Linux

2.10.2 Programación mixta C-asm Linux

Herramientas

Los programas se desarrollarán en C y en ensamblador. El ensamblador se utilizará básicamente para realizar ciertas optimizaciones en el código C (dentro del programa en C, "in line "). Las prácticas se realizarán con software de dominio público, en concreto con el compilador gcc bajo el sistema Linux, o bien con el paquete djgpp que implementa gcc en DOS e incluye además el entorno de desarrollo rhide para DOS.

Este paquete se puede encontrar en <http://www.delorie.com/djgpp>.

Ensamblador en GNU

El compilador gcc (y el djgpp) utiliza para el ensamblador la sintaxis de AT&T (GAS Gnu ASsembler). Esta tiene pequeñas diferencias con respecto a la sintaxis estándar de Intel (usada en TASM, MASM, etc). Las principales diferencias se detallan en la siguiente tabla (ver el apéndice I para más detalle):

Tabla 8

Diferencias entre sintaxis.

DIFERENCIAS	AT&T (Motorola)	Intel
1. En AT&T, a los nombres de los registros se les añade el prefijo %.	%eax	eax
2. En AT&T, el destino se coloca a la derecha y el fuente a la izquierda (en Intel es al contrario).	movl %eax, %ebx	mov ebx, eax
3. En AT&T, a los valores inmediatos se les añade el prefijo \$.	movl \$var, %eax movl \$0xf02, %ebx	mov eax, offset var mov ebx, 0xf02h
4. En AT&T, el tamaño del resultado se especifica con sufijos (b, w o l) en las instrucciones (en Intel cuando hay ambigüedad se utiliza byte ptr, word ptr o dword ptr).	movb var, %ah movw %bx, %ax movl %ebx, %eax	mov ah, byte ptr var mov ax, bx mov eax, ebx
5. Direccionamiento a memoria.	desplazamiento(base, índice, escala) movl array(, %eax, 4), %edx movl (%ebx), %eax movl 3(%ebx), %eax	[base+índice*escala+desplazamiento] mov edx, array[eax*4] mov eax, [ebx] mov eax, [ebx+3]
6. Salto lejano.	lcall \$sección, \$offset ljmp \$sección, \$offset lret \$V	call far sección:offset jmp far sección:offset ret far V
7. Varían los nemotécnicos de algunas instrucciones.	movswl %ax, %ecx movzbl %ah, %cx cwtl cwtd cltd	movsx ecx, ax movzx cx, ah cbw cwde cwtl cdq

En la página web del DJGPP podemos encontrar una guía detallada sobre la sintaxis AT&T, y ejemplos de ensamblador en línea:

http://www.delorie.com/djgpp/doc/brennan/brennan_att_inline_djgpp.html

Ensamblador en línea en gcc

A continuación, detallaremos la sintaxis utilizada para insertar código ensamblador en un programa en C. Para más información sobre la sintaxis utilizada por GAS se recomienda estudiar la página de manual (man as o bien info as).

Ensamblador en línea básico.

El formato básico para incluir instrucciones ensamblador es simple, y similar al utilizado por Borland:

```
asm ( "sentencias" )
```

Ejemplos:

- ejecutar la instrucción NOP
`asm ("nop")`
- deshabilitar interrupciones:
`asm ("cli")`
- y definiendo como una macro:
`#define disable() asm volatile ("cli")`

Se puede utilizar `__asm__` en lugar de `asm` si esta palabra reservada entra en conflicto con algún identificador del programa C.

Veamos cómo incluir varias líneas de ensamblador en un programa C:

```
__asm__ ("movl %eax, %ebx \n\t"  
        "movl $56, %esi \n\t"  
        "movl %ecx, $label(%edx , %ebx, $4) \n\t"  
        "movl %ah, (%ebx)");
```

Cada línea va encerrada entre comillas dobles (" "); el retorno de carro y el tabulador (`\n\t`) al final de cada línea se debe a que gcc pasa esas líneas tal cual a GAS, que espera una instrucción ensamblador por línea, y además indentada un tabulador.

Ensamblador en línea extendido

En el último ejemplo hemos hecho uso de los registros sin tener en cuenta si en instrucciones previas gcc tenía almacenado algún valor. Puesto que no queremos interferir con gcc en la asignación de registros a variables del programa C, en el bloque de instrucciones ensamblador deberemos indicar a qué variables queremos acceder y si es para lectura o escritura, en qué registros queremos guardarlas (y que gcc tenga esto en cuenta para preservar valores que pudieran perderse), etc.

De cualquier forma, lo más sencillo y seguro para evitar problemas será dejar que gcc elija los registros en los cuales guardar los valores que vayamos generando.

La sintaxis general del ensamblador en línea extendido será:

```
asm ( "sentencias"  
      : salidas  
      : entradas  
      : preservar_uso )
```

Donde:

- **sentencias:** Secuencia de instrucciones ensamblador separadas por caracteres "new line" ("\n"). Para mejorar aspecto en la salida como código ensamblador utilizar mejor "\n\t" como separación entre instrucciones.
- **Salidas:** Secuencia de variables gcc de salida entre paréntesis precedidas por las restricciones (y modificadores de éstas) a las que están sujetas (ver más adelante).
- **Entradas:** Secuencia de variables gcc de entrada entre paréntesis precedidas por las restricciones (y modificadores de éstas) a las que están sujetas.
- **Preservar_uso:** Para un correcto funcionamiento se debería informar al compilador de los recursos que se modifican dentro del código ensamblador "in line con el fin de que tenga cuidado en preservar su valor si resulta necesario. Los recursos a preservar se colocan separados por comas después de las entradas. Los recursos indicados en la línea de salida no se deben colocar en la lista para preservar, ya que el compilador ya sabe, por estar en la lista de salida, que se va a escribir sobre ellos. Se debería preservar los registros que se modifican dentro del código ensamblador (eax, ebx, ...) que no aparecen en la lista de salida. Si alguna instrucción ensamblador modifica memoria (y no se especifica en la línea de salida), se debe incluir para un correcto funcionamiento "memory" en la lista. Con "cc" se indica que se preserva el registro de estado. No se permite colocar en la lista de preserva registros asignados a entradas, sin embargo puede haber casos en los que el código ensamblador modifique estos registros (hay que tener cuidado con esto).

Ejemplo:

El siguiente código ensamblador en línea asigna el valor de la variable **a** a la variable **b** ($b = a$), y asigna el valor 4 a la variable **c** ($c = 4$).

```
asm    ("movl %2, %%eax \n\t \\  
        movl %%eax, %0 \n\t \\  
        movl $4, %%eax \n\t \\  
        movl %%eax, %1 \n\t"\  
        : "=g" (b), "=g" (c)  
        : "m" (a)  
        : "%eax" );
```

Sentencias:

```
movl %2, %%eax    \n\t  
movl %%eax, %0    \n\t  
movl $4, %%eax    \n\t  
movl %%eax, %1    \n\t
```

Dentro de las sentencias se hace referencia a las entradas y a las salidas con un número precedido de % (%0, %1, ..., %i, ..., %N). Cuando sólo se utilizan entradas (salidas) la asignación de número se debe realizar en orden de aparición de éstas en la lista de entradas (salidas) comenzando desde 0. Cuando se utilizan entradas y salidas, los números correlativos comenzando desde 0 se asignan primero a las salidas, y los siguientes números se asignan a las entradas, también de forma correlativa por orden de aparición en la lista. Por ejemplo, si se dispone de N variables, de las cuales K son de salida, la asignación por orden de aparición para las salidas sería: %0, %1, ..., %K-1; y la asignación para las entradas por orden de aparición de éstas sería: %K, %K+1, ..., %N-1.

Dentro del código "in line" los registros van precedidos de dos caracteres % en lugar de uno (para evitar confusión con las entradas y las salidas).

Salidas

Se especifica que se utilizan como salida las variables b y c. La restricción g le indica al compilador que puede escoger entre cualquier registro de propósito general o memoria para ubicar el resultado que va a pasar a b tras la ejecución de la secuencia

de instrucciones "in line ". El modificador = indica que se trata de una salida. En el ejemplo %0 se sustituye por la ubicación de b, y %1 por la ubicación de c.

Entradas

El contenido de la variable a es entrada al código ensamblador. La restricción m indica que la entrada al código ensamblador en la posición de %2 es memoria.

Preservar uso

En la lista aparece %eax. De esta forma se le indica al compilador que de los registros en los que puede asignar a entradas o a salidas (en este caso se lo puede asignar sólo a las salidas dado la restricción) excluya a eax, y además le indica que preserve su contenido si es necesario (puede que se haya modificado el contenido de este registro por una instrucción anterior al "in line" para ser utilizado por una instrucción posterior).

Ejemplo:

Veamos un programa C completo en el que se incluyen instrucciones ensamblador en línea:

En este ejemplo, se asigna el valor de la variable c a la variable a (a=c) utilizando eax como almacén intermedio; así mismo, se asigna el valor 4 a la variable b (b=4).

```
// COMPILAR:
//          gcc -Wall -g ej.c -o ej
#include <stdio.h>

#include <stdlib.h>

int main(void) {
    int a=7;
    int b=5;
    int c=0;

    __asm__      ("movl  %2,%%eax\n\t"
                  "movl  %%eax,%0\n\t"

                  "movl  $4,%%eax\n\t"
                  "movl  %%eax,%1\n\t"

                  : "=g" (a), "=g" (b)      /*salidas*/
                  : "m" (c)                 /*entradas*/
                  : "%eax"                  /*preservar valor*/
                  );

    printf("\n\n a=%d \n b=%d \n c=%d \n" , a, b, c );
    return 0;
}
```



Bibliografía

- Irvine, K. R. (2008). *Lenguaje ensamblador para computadoras basadas en Intel*. Mexico: Pearson Educacion.
- Orenga, M., & Manonellas, G. (2011). *Estructura de computadores*. Barcelona: Eureka Media.
- Stallings, W. (2005). *Organizacion y Arquitectura de Computadores*. Madrid: Pearson Educacion.
- Tanenbaum, S. A. (2000). *Organizacion de Computadoras: un enfoque estructurado*. Mexico: Prentice Hall.

